

Workflow-State Estimation from HoloLens Industrial Assembly Data

Technical Assignment — Part A

1 Introduction and Problem Formulation

This assignment uses multimodal data recorded with a Microsoft HoloLens 2 during an industrial manual assembly procedure (the IndustReal setting). The broader goal is multimodal operator support for human-centric manufacturing: using the sensors already present in an XR headset to understand what the operator is doing and to help them.

This report covers **Part A only**: procedure-step / workflow-state estimation. The task is to estimate, at each frame, how far the operator has progressed through the assembly, using sparse procedure-step (PSR) keyframe labels as the reference.

I formulate the target as a 9-bit *cumulative* multi-hot vector over the procedure steps

[3, 6, 9, 15, 18, 21, 24, 27, 30].

The labels are cumulative: once a step has been completed, its bit stays 1 for all later frames. So the vector represents the set of steps completed *so far* rather than a single current action.

This is useful because an XR system or a Human Digital Twin needs an estimate of where the operator is in the workflow before it can show the right instruction, confirm a completed step, or flag that something looks out of order.

2 Dataset and Modalities

Each recording provides RGB frames together with per-frame sensor logs: `gaze.csv` (2D gaze point and validity), `hands.csv` (hand joint coordinates and validity), `pose.csv` (head forward / position / up vectors), and `PSR_labels.csv` (the procedure-step keyframes).

I use the recordings `22_assy_0_1`, `22_assy_2_3`, `25_assy_0_1`, and `25_assy_2_1` for training, and `27_assy_0_1` as the held-out test recording. An important property of the data is that the PSR labels are *sparse keyframes* (a label only at the moment a step is completed), not dense per-frame labels.

I did not assume that all modalities had to be used equally. Instead I tested two representation directions for Part A:

- **Sensor-only features** built from gaze, hands, and pose.
- **Visual RGB features** from a frozen DINOv2 image encoder.

3 Label Construction

The sparse PSR keyframes are expanded into frame-level cumulative labels. For each frame t , a step bit is set to 1 if its keyframe has occurred at or before t , and stays 1 afterwards. From this I also define a simple scalar progress signal

$$\text{progress}_t = \frac{\sum_i y_{t,i}}{9},$$

where $y_{t,i}$ is the cumulative bit for step i at frame t .

One dataset-specific detail: step 12 is treated as an alias of step 9, because it corresponds to the alternative rear-chassis variant of the same stage. This is a simplifying assumption, but it avoids splitting and corrupting the step-9 training signal across two near-equivalent labels.

4 Approach 1: Sensor-Only Baseline

The first approach uses only the headset sensors. Each frame is turned into a 118-dimensional feature vector from the gaze point, the hand joint coordinates, the head pose vectors, and the corresponding validity flags.

For the model I used a `MultiOutputClassifier` wrapping a Random Forest, i.e. one binary classifier per step. A Random Forest is a reasonable first baseline here: the inputs are small tabular feature vectors, the relationships are nonlinear, it handles class imbalance through class weighting, and it produces probability outputs that are convenient for later temporal smoothing.

Because per-frame predictions are noisy, I tested several temporal fixes: sliding-window features, probability smoothing, a monotonic clamp, and sustained commitment (only commit a step after it stays above threshold for several frames). I keep this compact on purpose — the point is the engineering progression, not every variant.

The main finding is that the sensor-only Random Forest captures early and mid workflow state reasonably but struggles with late steps and with temporal stability. The interesting failure mode is that the hard temporal commitment did reduce the number of predicted transitions, but it could lock in early false positives. In other words, the problem was not only single-frame noise but *sustained early over-confidence* from the sensor features themselves, which no amount of smoothing removes.

5 Approach 2: RGB / DINOv2 Representation

The motivation for the second approach is that the sensor features do not directly show the state of the assembled object. The RGB frame, on the other hand, contains visual evidence of which parts, screws, and sub-assemblies are present.

I extract features with a frozen DINOv2 ViT-S/14 encoder (no fine-tuning). To keep computation manageable I sample every 5th frame and take the 384-dimensional CLS embedding per sampled frame. I then apply a rolling mean over the embeddings before classification, which gives a short temporal context for each prediction.

On top of these embeddings I compared three downstream classifiers:

- Logistic Regression — a linear probe;
- Random Forest — a nonlinear ensemble baseline;
- MLP — a small nonlinear neural head.

The comparison is intentionally compact because there are only four training recordings; a larger sweep would risk over-fitting the choice of model to one test recording. As with the sensor baseline, I tested probability smoothing and sustained commitment as temporal post-processing.

The final selected Part A method is **full-frame DINOv2 embeddings + MLP + probability smoothing** with window $W = 15$ and threshold 0.6. Sustained commitment was tested but *not* chosen as the main method: a commit-frame analysis showed that it locked in steps too early and missed step 27, so the stability it gave was not worth the timing errors it introduced.

6 Results and Observations

Table 1 compares the sensor baseline with the DINOv2 variants on the test recording. Because the two approaches use different input representations, this should be read as a **prototype-level representation comparison, not a strictly controlled classifier benchmark**.

Method	Macro F1	Hamming	Exact Match	Progress MAE	Transitions
Sensor RF + prob. smoothing	0.545	0.194	0.208	0.180	156
DINOv2 RF (raw)	0.598	0.162	0.298	0.152	30
DINOv2 MLP (raw)	0.704	0.166	0.452	0.151	81
DINOv2 MLP + prob. smoothing	0.801	0.136	0.478	0.128	31

Table 1: Part A test results on `27_assy_0_1`. Higher is better for Macro F1 and Exact Match; lower is better for Hamming Loss and Progress MAE. The true number of step transitions in the recording is small, so very large transition counts indicate temporal instability.

The improvement is clearest on the late steps. With the sensor baseline, step 27 reaches only $F1 = 0.218$ and step 30 only $F1 = 0.077$. With the DINOv2 MLP + smoothing method these rise to $F1 = 0.889$ (step 27) and $F1 = 0.764$ (step 30). Steps 21 and 24 remain difficult for all methods, mainly because they occur very late in the test sequence and have very few positive frames.

Main observations:

- The RGB / DINOv2 representation gave a clearly better signal than the gaze/hand/pose sensors for late workflow state.
- The MLP was the best of the three classifiers on the DINOv2 embeddings.
- Probability smoothing improved temporal stability (far fewer predicted transitions) without making errors permanent, which was the problem with hard sustained commitment.
- Transition timing is still imperfect, so this remains a prototype rather than a production-ready procedure recognizer.

7 Practical Relevance

Even as a prototype, a cumulative workflow-state estimate has several uses for operator support:

- **XR guidance:** knowing the current state tells the system which instruction step to show next or which completed step to confirm.

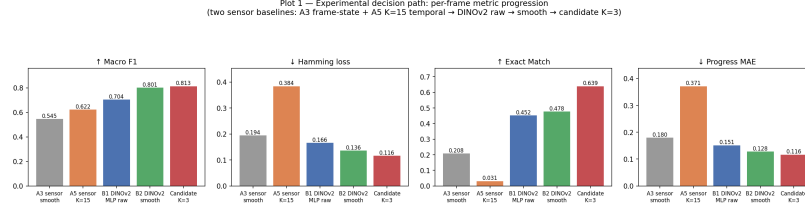


Figure 1: Metric progression across the approaches and temporal post-processing.

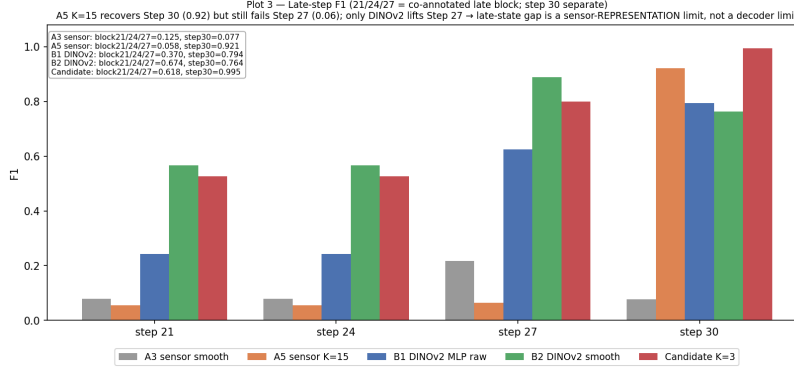


Figure 2: Late-step F1: the RGB representation recovers steps the sensor baseline misses.

- **Human Digital Twin:** the cumulative vector is a compact representation of the operator’s current task progress.
- **Operator-support analytics:** steps that are consistently late or unstable in the prediction point to difficult stages or possible workflow deviations.
- **Training:** comparing expected workflow progress with observed progress can highlight where new operators fall behind.

8 Limitations

- Only four training recordings and one test recording, so the results are initial evidence on a limited dataset.
- PSR labels are sparse keyframes and require a pseudo / frame-level cumulative expansion to train per-frame classifiers.
- The cumulative state ignores fine-grained action detail within a step.
- Step ordering and the step-12 → step-9 alias are simplifications.
- Late steps are highly imbalanced, especially steps 21, 24, and 27.
- The classifiers can still predict early false completions.
- Probability smoothing improves stability but does not fully solve transition timing.
- The DINOv2 embeddings are frozen and not specialised for industrial assembly.
- The prototype is offline and not yet deployed in real-time XR.

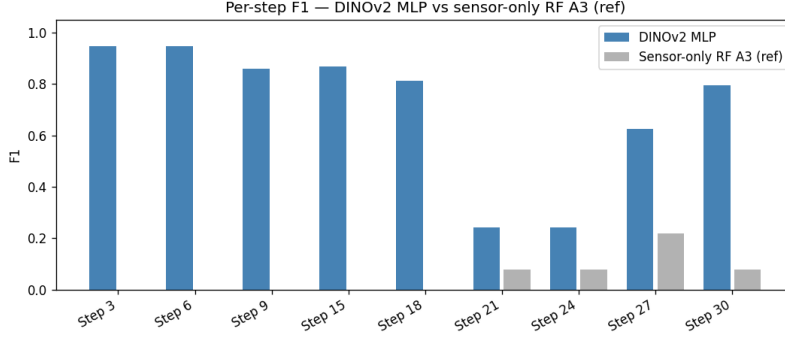


Figure 3: Per-step F1 for the DINOv2 MLP method.

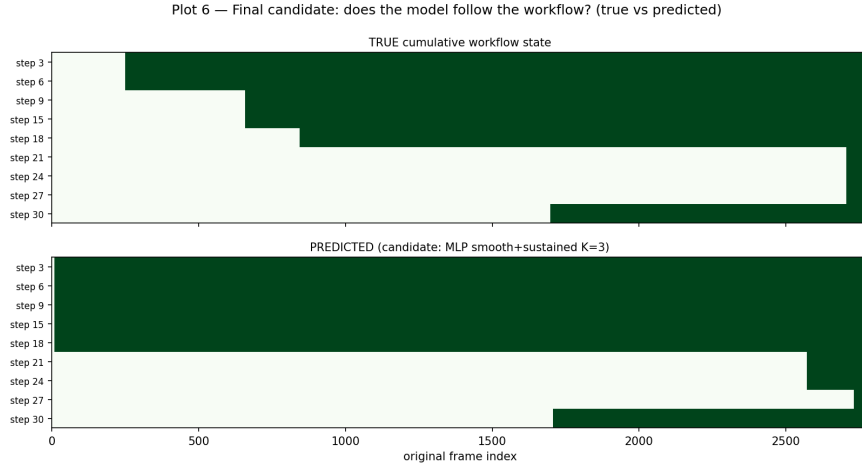


Figure 4: Predicted cumulative workflow state over time for the final method, compared against the ground-truth labels.

9 Conclusion and Future Work

This prototype estimates cumulative workflow state from HoloLens industrial assembly data. The sensor-only features give a useful baseline but are limited for late workflow state, where the visual appearance of the assembly matters most. Frozen DINOv2 RGB embeddings with an MLP and probability smoothing gave the strongest result and suggest that visual evidence is the more informative representation for this task. The approach is a feasible initial workflow-state estimator for XR guidance or Human Digital Twin applications, while still being limited by the small dataset and imperfect labels.

For future work, the natural next steps are a sequence model (LSTM or Transformer) instead of per-frame classification, a monotonic dynamic-programming decoder that enforces valid cumulative ordering, multimodal fusion of RGB and sensor features, a full-frame + hand-crop fusion to combine global and local visual evidence, and more recordings together with real-time XR integration.